

Satbayev University · CityU Programme

CSE6624

Introduction to Web Programming

Practice & Lab Workbook

Autumn 2026–2027 · Saturdays 15:25–18:20 · Room GUK714

Instructor (lecture + practice + lab): **Temirlan Shoiynbek**

shoinbektima@gmail.com

What this PDF is. One complete student workbook for every Practice (≈50–60 min) and Lab (≈50–60 min) of the course. Follow the numbered steps. Do not guess which button to click — every action is written out. Keep this file open on a second screen or phone while you work.

Audience. Year-1 absolute beginners. English is the course language. Short Russian tip boxes appear only for install pitfalls.

Out of scope. React, Node.js, npm apps, databases, PHP. Client-side HTML, CSS, JavaScript, Bootstrap only.

Version 1.0 · Generated for Autumn 2026–2027

Contents

- Chapter 0 — Setup once (Windows + macOS)
- How to submit · Done criteria · AI policy
- Practice & Lab weeks 01–15 (matching the syllabus)
- Attestation 1 note (after Week 08)
- End-of-term / exam project defence checklist

Weekly map: W1 WWW/HTTP + first HTML · W2 Semantic HTML5 · W3 Text/lists/links/media · W4 Tables & forms · W5 CSS cascade & box model · W6 Flexbox · W7 Grid + media queries · W8 Project workshop · Attestation 1 · W9 Bootstrap · W10 JS basics · W11 Functions/arrays/objects · W12 DOM & events · W13 localStorage + fetch · W14 Accessibility & polish · W15 Defence prep · Exam checklist.

Chapter 0 — Setup once (do this before Week 01)

Complete this chapter on your own laptop **before** the first Saturday, or in the first 20 minutes of Week 01 Practice. When finished, you can create a folder, open it in VS Code, write HTML, and preview it with Live Server.

0.1 Modern browser (Chrome or Edge)

1. Install **Google Chrome** from <https://www.google.com/chrome/> or use Microsoft Edge (already on Windows) / Safari is OK on macOS but Chrome is preferred for DevTools demos.
2. Open the browser. In the address bar type **chrome://version** (Chrome) or **edge://version** (Edge). Confirm it opens.
3. Pin the browser to your taskbar / Dock for fast access on Saturdays.

Tip — browser

If a university PC blocks installs: use portable Chrome or the Edge already on the classroom machine. Do not use Internet Explorer.

0.2 VS Code

1. Open <https://code.visualstudio.com/> in your browser.
2. Click the big blue **Download** button for your OS (Windows / macOS / Linux).
3. Run the installer. On Windows: accept defaults, enable “Add to PATH” if offered, finish.
4. On macOS: drag Visual Studio Code into Applications, then open it once (right-click → Open if Gatekeeper warns).
5. Launch VS Code. You should see a Welcome tab. Close it for now.

Tip — VS Code will not open (macOS)

System Settings → Privacy & Security → Open Anyway. Or: right-click the app icon → Open.

0.3 Extensions (exact Marketplace names)

In VS Code, click the **Extensions** icon in the left Activity Bar (four squares; shortcut **Ctrl+Shift+X** on Windows/Linux, **Cmd+Shift+X** on macOS). In the search box, type each name below exactly, click the extension, then click **Install**.

- **Live Server** — publisher Ritwick Dey — Marketplace:

<https://marketplace.visualstudio.com/items?itemName=ritwickdey.LiveServer>

- **Prettier - Code formatter** — publisher Prettier —

<https://marketplace.visualstudio.com/items?itemName=esbenp.prettier-vscode>

- **ESLint** — publisher Microsoft — <https://marketplace.visualstudio.com/items?itemName=dbaeumer.vscode-eslint>

1. After Install, the button becomes **Uninstall** / shows a gear — that means it is installed.
2. Optional but useful: **HTML CSS Support** and **Auto Rename Tag** (search those names and Install).

0.4 Git + GitHub account

1. Install Git from <https://git-scm.com/>. Windows: use the 64-bit installer; keep defaults (editor can be VS Code; default branch name **main** is fine).
2. Open a terminal: in VS Code menu **Terminal** → **New Terminal**.
3. Type **git --version** and press Enter. You should see a version number (e.g. git version 2.x).
4. Create a free GitHub account at <https://github.com/signup> if you do not have one. Verify your email.
5. You do not need to push every week, but GitHub is useful for your final portfolio and optional Pages hosting.

Tip — Git on Windows

After installing Git, fully quit and reopen VS Code so Terminal can see git. If the command is still not found, reboot once.

0.5 Create a course folder and open it in VS Code

1. Decide a parent location, e.g. Desktop or Documents.
2. Windows: open File Explorer → go to Desktop → right-click empty space → **New** → **Folder** → name it **CSE6624**.
3. macOS: Finder → Desktop → File → New Folder → name it **CSE6624**.
4. In VS Code: **File** → **Open Folder...** (macOS: Open...) → select the **CSE6624** folder → Open.
5. Confirm the Explorer sidebar (top-left files icon) shows your empty CSE6624 folder.

0.6 New File, Save, Live Server Go Live

1. In Explorer, click the **New File** icon (page with +) next to the folder name, OR right-click the folder → New File.
2. Type the file name exactly: **index.html** and press Enter.
3. Paste a minimal HTML skeleton (see snippet below). Press **Ctrl+S** / **Cmd+S** to Save.
4. Bottom-right of VS Code status bar: click **Go Live** (added by Live Server). If you do not see it: right-click **index.html** in Explorer → **Open with Live Server**.
5. Your browser opens a local URL like **http://127.0.0.1:5500/index.html**. You should see your page.
6. To stop: click **Port: 5500** in the status bar, or close the browser tab and click Go Live again later.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Setup test</title>
</head>
<body>
  <h1>Setup OK</h1>
  <p>If you see this via Live Server, you are ready for Week 01.</p>
</body>
</html>
```

Do not rely on double-click only

Double-clicking index.html opens a file:// URL. Some later features (fetch of local JSON, modules) fail there. Always prefer Live Server (http://127.0.0.1:...).

0.7 How to take a screenshot for submission

- **Windows:** press **Win+Shift+S** → select the area (browser + VS Code if asked) → paste into Paint or Word → Save as PNG. Or use Snipping Tool from the Start menu.
- **macOS:** press **Cmd+Shift+4** → drag to select → PNG appears on Desktop. Or **Cmd+Shift+5** for options.
- Name screenshots clearly, e.g. **W01_practice_liveserver.png**.
- Include in the shot: the browser address bar showing **http://127.0.0.1:...** and the visible page content.

0.8 Folder naming (use every week)

Replace Surname and Name with your Latin transliteration (as in your student ID / email). Replace XX with the two-digit week number.

```
CSE6624_W01_Practice-Shoiynbek_Temirlan
CSE6624_W01_Lab-Shoiynbek_Temirlan
CSE6624_W12_Practice-Ivanov_Ivan
CSE6624_W12_Lab-Ivanov_Ivan
```

- Put each week's Practice and Lab in separate folders under your main CSE6624 folder.
- When submitting: zip the folder (right-click → Compress / Send to compressed folder). Zip name = same as folder name + .zip.
- Do not submit only screenshots without code, unless the teacher asks for screenshot-only proof.

Chapter 0 Done criteria

- VS Code opens your CSE6624 folder
- Live Server, Prettier, ESLint installed
- git --version works in VS Code terminal
- A test index.html opens at <http://127.0.0.1:5500/>...

How to submit · Done criteria · AI policy

How to submit (every Practice / Lab)

1. Finish the checklist for that session.
2. Save all files (Ctrl+S / Cmd+S). Confirm Live Server still shows the latest version (refresh browser with F5).
3. Zip your folder: Windows → right-click folder → **Compress to ZIP** / **Send to** → **Compressed folder**. macOS → right-click → **Compress**.
4. Upload the zip to the channel announced by the instructor (LMS / Google Classroom / email — follow the week's announcement).
5. If the teacher checks in class: keep the browser tab open and be ready to explain one change you made.

Done criteria language (what “finished” means)

You are **Done** for a session when **all** of the following are true:

- Every checkbox in the Deliverable checklist is completed.
- The page opens via Live Server without missing files (no broken CSS/JS paths).
- You can explain, in one minute, what you changed and why.
- If AI was used: you can modify a line the teacher points to without panic.
- Folder naming matches the pattern in Chapter 0.

Soft grading philosophy

This course uses soft grading for weekly practice/lab: effort, completeness, and ability to explain matter more than pixel-perfect design. Missing work or empty folders are not “soft.” Late work: follow the instructor's announcement.

AI policy (allowed with conditions)

- **Allowed:** ChatGPT, Copilot, Claude, etc. to draft HTML/CSS/JS, explain errors, suggest fixes.
- **Required:** You must be able to **explain** every part you submit and **change** it live (rename a class, alter a color, add a console.log).
- **Disclose:** In README.md add a short line: “AI used for: ... / I rewrote: ...”.
- **Not allowed:** Submitting code you cannot explain; generating a whole project and never reading it; using AI during a closed-book attestation if the instructor forbids it that day.

Defence rule

At Attestation and Exam defence, the teacher may ask you to edit your own code. If you cannot, the AI help did not count as learning — grades will reflect that.

Classroom rhythm (Saturdays)

- Lecture ≈ 50 min (slides + demos).
- Practice ≈ 50–60 min (this workbook — Practice XX).
- Lab ≈ 50–60 min (this workbook — Lab XX; harder / longer; builds on Practice).
- Ask questions after you tried the “If it does not work” section for 3–5 minutes.

PRACTICE 01 — WWW / HTTP + toolchain + first HTML

Timebox: ≈50–55 minutes · Folder: `CSE6624\_W01\_Practice\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Prove you can create a personal hello page, open it with Live Server, and show a local `http://` URL. Connect the idea “browser requests a document” to what you just did.

Deliverable checklist

- Folder named `CSE6624_W01_Practice_Surname_Name`
- File `index.html` with your real name in an `<h1>`
- Short paragraph: faculty / year / group
- Link to Satbayev University website (<https://satbayev.university>)
- Opened via Live Server (address bar shows `http://127.0.0.1:...` or `http://localhost:...`)
- Screenshot of browser with address bar visible OR note of the URL written in `README.txt`

Click-by-click steps

1. Create folder `CSE6624_W01_Practice_Surname_Name` inside your `CSE6624` directory (Chapter 0).
2. VS Code: File → Open Folder → select that practice folder.
3. New File → name it `index.html` → Enter.
4. Type (or paste) the starter HTML below. Replace YOUR NAME, faculty, year, group.
5. Save (Ctrl+S / Cmd+S).
6. Right-click `index.html` → Open with Live Server (or click Go Live).
7. Confirm the heading shows your name. Click the university link — a new tab should open the real site.
8. Take a screenshot including the address bar. Save it as `W01_practice_liveserver.png` in the same folder (optional but recommended).
9. Write one sentence in a new file `notes.txt`: “HTTP is ... (request/response in your words).”

Starter snippets (copy-paste, then edit)

HTML snippet

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Week 01 Practice – Your Name</title>
</head>
<body>
  <h1>Hello, I am YOUR NAME</h1>
  <p>I study at Satbayev University, Faculty of ____, year __, group ____.</p>
  <p>
    <a href="https://satbayev.university" target="_blank" rel="noopener">
      University website
    </a>
  </p>
  <hr />
  <p>This page was opened with Live Server for CSE6624.</p>
</body>
</html>
```

If it does not work

- Go Live missing → install Live Server extension (Chapter 0); reload VS Code window (Ctrl+Shift+P → “Reload Window”).
- Page blank → check you saved `index.html`; refresh browser (F5).

- Link does nothing → check https:// and quotes around the URL; look for red underlines in VS Code.
- Wrong folder opened → File → Open Folder again; Explorer must show only this week's files.

What to show the teacher (Done criteria)

Done when

- Browser tab with your name visible
- Address bar starts with http://127.0.0.1 or http://localhost
- You can say in one sentence what Live Server does

LAB 01 — Three-file site skeleton (HTML + CSS + JS)

Timebox: ≈50–60 minutes · Folder: `CSE6624\_W01\_Lab\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Build a mini project with separate css/styles.css and js/main.js linked correctly. Prove styles and a button click work.

Deliverable checklist

- Folder CSE6624_W01_Lab_Surname_Name
- index.html with <header>, <main>, <footer>
- css/styles.css linked with a stylesheet link tag (rel=stylesheet, href=css/styles.css)
- js/main.js linked with <script src="js/main.js"></script> before </body>
- CSS visibly changes background or text color
- Button click updates #message text; Console shows a console.log message
- README.md with your name and “Open with Live Server”

Click-by-click steps

1. Create folder CSE6624_W01_Lab_Surname_Name. Open it in VS Code.
2. Create subfolders: click New Folder → css ; New Folder → js.
3. Create files: index.html (root), css/styles.css, js/main.js, README.md.
4. Paste the HTML/CSS/JS snippets below into the correct files. Edit the TODO comments away.
5. Save all. Open with Live Server.
6. Confirm background color from CSS is visible.
7. Open DevTools: F12 (Windows) or Cmd+Option+I (macOS) → Console tab.
8. Click the button. Confirm page text changes and Console shows your log line.
9. Fill README.md: name, group, how to run.

Starter snippets (copy-paste, then edit)

HTML snippet

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Week 01 Lab Skeleton</title>
  <link rel="stylesheet" href="css/styles.css" />
</head>
<body>
  <header>
    <h1>My Lab Skeleton</h1>
  </header>
  <main>
    <p id="message">JS will change this text.</p>
    <button id="go" type="button">Update message</button>
  </main>
  <footer>
    <p>CSE6624 Week 01 · Your Name</p>
  </footer>
  <script src="js/main.js"></script>
</body>
</html>
```

CSS snippet

```
/* css/styles.css */
body {
  font-family: system-ui, sans-serif;
  margin: 0;
  background: #eef2ff;
  color: #1a202c;
}
header, footer {
  background: #1a365d;
  color: #fff;
  padding: 0.75rem 1.25rem;
}
main {
  padding: 1.25rem;
}
button {
  padding: 0.5rem 1rem;
  cursor: pointer;
}
```

JAVASCRIPT snippet

```
// js/main.js
console.log("Week 01 lab script loaded");

const message = document.getElementById("message");
const button = document.getElementById("go");

button.addEventListener("click", function () {
  message.textContent = "Button works. Hello from JavaScript!";
  console.log("Button clicked");
});
```

If it does not work

- CSS not applied → check href path is css/styles.css (not /css/...); file must be inside css/ folder.
- JS error “Cannot read properties of null” → id spelling must match getElementById; script must be after the HTML elements (before </body>).
- Console empty on load → hard refresh Ctrl+Shift+R; confirm script src path.

What to show the teacher (Done criteria)

Done when

- Colored page from CSS
- Button changes the paragraph
- Console log visible in DevTools
- README.md exists

Why Lab ≠ Practice

Practice = one HTML file hello. Lab = structured project with linked CSS + JS. Same theme, deeper skill.

PRACTICE 02 — Semantic HTML5 landmarks

Timebox: ≈50–55 minutes · Folder: `CSE6624\_W02\_Practice\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Rebuild a page using semantic landmarks: header, nav, main, article, section, aside, footer. Avoid a page made only of <div> soup.

Deliverable checklist

- One index.html with DOCTYPE, lang, charset, viewport
- Uses <header>, <nav>, <main>, <article> or <section>, <footer>
- Nav has at least 3 placeholder links (href="#...")
- Page title and h1 describe a campus topic (club / dorm / library tip)
- No CSS required this practice (structure only)

Click-by-click steps

1. Create and open folder CSE6624_W02_Practice_Surname_Name.
2. Create index.html. Paste the starter below.
3. Replace the topic with your own (e.g. “Robotics Club Guide”).
4. Add a second <section> with a short paragraph and an <h2>.
5. Add an <aside> with “Office hours” or “Quick tip”.
6. Save → Live Server → view. In DevTools Elements panel, expand the DOM and point to each landmark.
7. Self-check: would a screen reader find main content easily? (main should wrap the primary content once.)

Starter snippets (copy-paste, then edit)

HTML snippet

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Campus Club Guide – Practice W02</title>
</head>
<body>
  <header>
    <h1>Robotics Club Guide</h1>
    <nav aria-label="Primary">
      <ul>
        <li><a href="#about">About</a></li>
        <li><a href="#meetings">Meetings</a></li>
        <li><a href="#join">Join</a></li>
      </ul>
    </nav>
  </header>
  <main>
    <article id="about">
      <h2>About</h2>
      <p>We build small robots and learn sensors on Saturdays.</p>
    </article>
    <section id="meetings">
      <h2>Meetings</h2>
      <p>Every other Saturday, room TBA.</p>
    </section>
    <aside>
      <h2>Quick tip</h2>
      <p>Bring a laptop with VS Code installed.</p>
    </aside>
  </main>
  <footer>
    <p>CSE6624 · Your Name · Week 02 Practice</p>
  </footer>
</body>
</html>
```

If it does not work

- Validation feel: each heading level should not skip oddly (h1 then h2, not h1 then h4).
- Multiple <main> is wrong — use exactly one <main> per page.
- Nav links jump nowhere → ids on targets must match href="#about" etc.

What to show the teacher (Done criteria)

Done when

- You can point to header/nav/main/footer in the Elements panel
- Topic text is yours (not left as Robotics if you changed it)

LAB 02 — Two-page semantic mini-site

Timebox: ≈55–60 minutes · Folder: `CSE6624\_W02\_Lab\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Create index.html and about.html that share the same nav structure and link to each other with relative paths.

Deliverable checklist

- Files: index.html, about.html, README.md
- Both pages use semantic landmarks
- Nav links work both ways (index ↔ about)
- Relative paths only (no file:/// hardcoding)
- Each page has a unique <title> and <h1>

Click-by-click steps

1. Create lab folder; open in VS Code.
2. Create index.html (home) and about.html.
3. Copy semantic structure to both; change main content per page.
4. On index: About. On about: Home.
5. Live Server from index — click About — confirm URL ends with about.html.
6. Use browser Back; confirm Home works.
7. README: name + list of files + how to open.

Starter snippets (copy-paste, then edit)

HTML snippet

```
<!-- Nav fragment to reuse on BOTH pages -->
<nav aria-label="Primary">
  <ul>
    <li><a href="index.html">Home</a></li>
    <li><a href="about.html">About</a></li>
  </ul>
</nav>
```

If it does not work

- Link downloads or 404 → filenames must match exactly (case-sensitive on some systems).
- Opened wrong page via double-click → use Live Server from the folder root.

What to show the teacher (Done criteria)

Done when

- Click from Home to About and back in front of the teacher
- Explain what a relative path means in one sentence

PRACTICE 03 — Text, lists, links, images, media

Timebox: ≈50–55 minutes · Folder: `CSE6624\_W03\_Practice\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Write a short article page with headings, lists, internal/external links, and at least one image with alt text. Understand relative vs absolute URLs for assets.

Deliverable checklist

- index.html article with h1 + at least two h2
- One and one
- One external link (<https://...>) and one internal link (#id)
- images/ folder with at least one image;
- alt text describes the image meaningfully (not “image1”)

Click-by-click steps

1. Create practice folder; open in VS Code.
2. Create images/ subfolder. Add a small JPG/PNG (your photo of campus, or a free image from <https://picsum.photos/> — download and save locally as images/campus.jpg).
3. Create index.html. Build an article “How I study on Saturdays” (or your topic).
4. Add a checklist as and “3 steps I take” as .
5. Insert .
6. Add link to MDN HTML intro: https://developer.mozilla.org/en-US/docs/Learn/HTML/Introduction_to_HTML.
7. Live Server → confirm image loads (not broken-icon).

Starter snippets (copy-paste, then edit)

HTML snippet

```
<main>
  <article>
    <h1>How I study on Saturdays</h1>
    <p>After lecture I open this workbook and follow Practice steps.</p>
    <h2>Tools I keep open</h2>
    <ul>
      <li>VS Code</li>
      <li>Chrome DevTools</li>
      <li>This PDF workbook</li>
    </ul>
    <h2>Three steps</h2>
    <ol>
      <li>Read the goal</li>
      <li>Code the checklist</li>
      <li>Show Live Server to the teacher</li>
    </ol>
    <p></p>
    <p><a href="https://developer.mozilla.org/en-US/docs/Learn/HTML/Introduction_to_HTML">MDN: Intro to...</a></p>
  </article>
</main>
```

If it does not work

- Broken image → path must be relative to the HTML file; check spelling and folder name images/.
- Huge image → add width="480" or resize the file; do not commit 10 MB photos.
- alt empty → accessibility fail for this course; write a short description.

What to show the teacher (Done criteria)

Done when

- Image visible
- Lists render with bullets/numbers
- You can explain src vs alt

LAB 03 — Media gallery page + figure/figcaption

Timebox: ≈55–60 minutes · Folder: `CSE6624\_W03\_Lab\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Build a small gallery using `<figure>` and `<figcaption>`, plus an optional audio or video element (or a clearly labeled placeholder if no media file).

Deliverable checklist

- At least 3 figures with images + captions
- Consistent images/ paths
- One page title/h1 for a “Campus Media Board”
- README lists image sources (own photo / placeholder site)
- Optional: `<video controls>` or `<audio controls>` with a short local file, OR a comment explaining why omitted

Click-by-click steps

1. Create lab folder + images/ with 3 images (can be different crops or picsum downloads).
2. Write index.html with header/main/footer.
3. For each image wrap in `<figure>...<figcaption>...</figcaption></figure>`.
4. Add a short intro paragraph and a list of “photo credits”.
5. Test all images on Live Server.
6. Write README.md with credits and how to open.

Starter snippets (copy-paste, then edit)

HTML snippet

```
<figure>
  
  <figcaption>Lab room – practice hour.</figcaption>
</figure>
```

If it does not work

- Mixed path styles (Images vs images) break on Linux servers — use lowercase folder names.
- Spaces in filenames cause pain — prefer campus-1.jpg not My Photo.jpg.

What to show the teacher (Done criteria)

Done when

- Three figures with captions
- No broken images
- README credits present

PRACTICE 04 — Tables and forms basics

Timebox: ≈50–55 minutes · Folder: `CSE6624\_W04\_Practice\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Build a small schedule table and a contact form with labels, name/email/message, and required fields.

Deliverable checklist

- Table with thead/tbody, at least 3 data rows (day, time, room)
- Form with `<label for="...">` matching input id
- input type=text, email; textarea; submit button
- required on name and email
- form method="get" action="#" is OK for this week (no backend)

Click-by-click steps

1. Create practice folder; index.html.
2. Add `<h1>Club Schedule</h1>` and a table (see snippet).
3. Add `<h2>Contact</h2>` and a form with labels.
4. Live Server → try Submit with empty fields — browser native validation should block.
5. Fill valid email and submit — URL may show query string; that is OK for GET demos.

Starter snippets (copy-paste, then edit)

HTML snippet

```
<h1>Club Schedule</h1>
<table border="1">
  <thead>
    <tr><th scope="col">Day</th><th scope="col">Time</th><th scope="col">Room</th></tr>
  </thead>
  <tbody>
    <tr><td>Saturday</td><td>15:25</td><td>GUK714</td></tr>
    <tr><td>Saturday</td><td>16:30</td><td>GUK714</td></tr>
    <tr><td>Saturday</td><td>17:30</td><td>GUK714</td></tr>
  </tbody>
</table>

<h2>Contact</h2>
<form method="get" action="#">
  <p>
    <label for="name">Name</label><br />
    <input id="name" name="name" type="text" required />
  </p>
  <p>
    <label for="email">Email</label><br />
    <input id="email" name="email" type="email" required />
  </p>
  <p>
    <label for="msg">Message</label><br />
    <textarea id="msg" name="msg" rows="4" cols="40"></textarea>
  </p>
  <button type="submit">Send</button>
</form>
```

If it does not work

- Label click does not focus input → `for="name"` must equal `id="name"`.
- Email accepts garbage → use `type="email"` (browser checks basic format).
- Table looks cramped → `border="1"` is OK for learning; CSS comes next weeks.

What to show the teacher (Done criteria)

Done when

- Table with header row
- Empty submit blocked by required
- You can point to label/for/id pairing

LAB 04 — Registration form + data table

Timebox: ≈55–60 minutes · Folder: `CSE6624\_W04\_Lab\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Combine a richer form (select, radio, checkbox) with a results/info table on one page. Emphasize accessible labels and fieldset/legend for radio groups.

Deliverable checklist

- fieldset/legend for a radio group (e.g. year of study)
- One `<select>` with ≥ 3 options
- One checkbox (e.g. “I agree”)
- Info table (≥ 4 rows) related to the topic
- README with your name

Click-by-click steps

1. Create lab folder; plan topic (e.g. “Workshop registration”).
2. Build the info table first (static data).
3. Build the form under it with fieldset for Study year radios.
4. Add select for preferred track (HTML / CSS / JS).
5. Add checkbox required for agreement.
6. Test validation paths; screenshot optional.
7. README: what each control is for.

Starter snippets (copy-paste, then edit)

HTML snippet

```
<fieldset>
  <legend>Year of study</legend>
  <label><input type="radio" name="year" value="1" required /> 1</label>
  <label><input type="radio" name="year" value="2" /> 2</label>
  <label><input type="radio" name="year" value="3" /> 3</label>
</fieldset>
<p>
  <label for="track">Track</label>
  <select id="track" name="track">
    <option value="html">HTML</option>
    <option value="css">CSS</option>
    <option value="js">JavaScript</option>
  </select>
</p>
<p>
  <label><input type="checkbox" name="agree" required /> I agree to the club rules</label>
</p>
```

If it does not work

- Radios must share the same `name="year"` to be one group.
- Only one radio required attribute is enough for the group in most browsers.

What to show the teacher (Done criteria)

Done when

- fieldset visible
- select + checkbox work
- table + form on one page

PRACTICE 05 — CSS selectors, cascade, box model

Timebox: ≈50–55 minutes · Folder: `CSE6624\_W05\_Practice\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Link an external stylesheet and style a simple page using element, class, and descendant selectors. Practice margin, padding, border (box model).

Deliverable checklist

- index.html + styles.css linked
- At least one class selector (.card) and one element selector (body or h1)
- Visible padding and border on .card
- nav links styled (color + remove default underline optionally)
- You can explain which rule wins if two rules conflict (specificity idea)

Click-by-click steps

1. Create folder; files index.html and styles.css.
2. Paste HTML with header.nav and three .card articles.
3. In styles.css set body font and background; style .card with padding, border, margin, background.
4. Style .nav a { color: ...; text-decoration: none; margin-right: 1rem; }.
5. Add a more specific rule, e.g. header .nav a { font-weight: bold; } — observe cascade.
6. Live Server → use DevTools → select a card → Box Model diagram (margin/border/padding).

Starter snippets (copy-paste, then edit)

HTML snippet

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Week 05 Practice</title>
  <link rel="stylesheet" href="styles.css" />
</head>
<body>
  <header class="top">
    <h1>Week 05 – CSS Basics</h1>
    <nav class="nav">
      <a href="#">Home</a>
      <a href="#">About</a>
    </nav>
  </header>
  <main class="wrap">
    <section class="cards">
      <article class="card">Card A – padding and border</article>
      <article class="card">Card B – box model</article>
      <article class="card">Card C – cascade</article>
    </section>
  </main>
</body>
</html>
```

CSS snippet

```
body {  
  margin: 0;  
  font-family: Georgia, "Times New Roman", serif;  
  background: #f7fafc;  
  color: #1a202c;  
}  
.top {  
  background: #1a365d;  
  color: #fff;  
  padding: 1rem 1.25rem;  
}  
.nav a {  
  color: #bee3f8;  
  text-decoration: none;  
  margin-right: 1rem;  
}  
header .nav a {  
  font-weight: bold;  
}  
.wrap { padding: 1.25rem; }  
.card {  
  background: #fff;  
  border: 2px solid #2b6cb0;  
  padding: 1rem;  
  margin-bottom: 0.75rem;  
  border-radius: 8px;  
}
```

If it does not work

- CSS ignored → link href filename must match; check Console/Network for 404.
- Confused by box size → in DevTools toggle box-sizing; try box-sizing: border-box on *.

What to show the teacher (Done criteria)

Done when

- Cards clearly padded with borders
- You open Box Model panel and name margin vs padding

LAB 05 — Themed two-column card layout (box model deep dive)

Timebox: ≈55–60 minutes · Folder: `CSE6624\_W05\_Lab\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Theme a fuller page: header, multi-card main area, footer. Use width/max-width, box-sizing, and spacing consistently.

Deliverable checklist

- styles.css uses `* { box-sizing: border-box; }`
- main max-width ~900px centered
- At least 3 `.panel` blocks with distinct padding
- Hover style on panels optional but recommended
- README.md

Click-by-click steps

1. Create lab folder with index.html + styles.css.
2. Build header/main/footer; main contains `.layout` with three `.panel` sections.
3. Center content: `main { max-width: 900px; margin: 0 auto; padding: 1rem; }`
4. Add `:hover` background change on `.panel`.
5. Experiment: change padding and watch Box Model.
6. Document in README your color choices in one sentence.

Starter snippets (copy-paste, then edit)

CSS snippet

```
* { box-sizing: border-box; }
main {
  max-width: 900px;
  margin: 0 auto;
  padding: 1rem;
}
.layout .panel {
  border: 1px solid #cbd5e0;
  padding: 1.25rem;
  margin-bottom: 1rem;
  background: #fff;
}
.layout .panel:hover {
  background: #ebf8ff;
}
```

If it does not work

- Horizontal scrollbar → usually width:100% + padding without border-box.

What to show the teacher (Done criteria)

Done when

- Centered layout
- Hover works
- box-sizing present

PRACTICE 06 — Flexbox — nav + cards row

Timebox: ≈50–55 minutes · Folder: `CSE6624\_W06\_Practice\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Use `display:flex` to build a horizontal nav and a wrapping row of cards.

Deliverable checklist

- `header.top` is flex with space-between (title left, nav right)
- `.cards` is flex with flex-wrap: wrap and gap
- `.card` has a flex basis (e.g. `flex: 1 1 200px`)
- Works with 3+ cards

Click-by-click steps

1. Start from Week 05 HTML structure (or paste starter).
2. In CSS set `.top { display: flex; align-items: center; justify-content: space-between; }`.
3. Set `.cards { display: flex; flex-wrap: wrap; gap: 1rem; }`.
4. Set `.card { flex: 1 1 200px; }` plus padding/border.
5. Resize the browser window — cards should wrap to the next line.
6. Optional: `.nav { display: flex; gap: 1rem; }`.

Starter snippets (copy-paste, then edit)

CSS snippet

```
.top {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0.75rem 1.25rem;
  background: #1a365d;
  color: #fff;
}
.nav { display: flex; gap: 1rem; }
.nav a { color: #bee3f8; text-decoration: none; }
.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
  padding: 1.25rem;
}
.card {
  flex: 1 1 200px;
  border: 1px solid #a0aec0;
  padding: 1rem;
  background: #fff;
}
```

If it does not work

- Items not in a row → parent needs `display:flex` (not only children).
- No wrap → add `flex-wrap: wrap`.
- Huge gaps → check gap vs margin double-spacing.

What to show the teacher (Done criteria)

Done when

- Nav on one row with title
- Cards wrap when window is narrow

LAB 06 — Product / club cards + flex footer

Timebox: ≈55–60 minutes · Folder: `CSE6624\_W06\_Lab\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Build a fuller flex page: hero strip, wrapping card grid-like row, footer with flex-spaced columns.

Deliverable checklist

- Hero section with flex alignment
- ≥4 cards in a wrapping flex row
- Footer with `display:flex; justify-content: space-between`
- Consistent gap usage
- README

Click-by-click steps

1. Create lab page about a “Campus Fair” with 4 booth cards.
2. Hero: `display:flex; align-items:center; min-height: 160px; padding`.
3. Cards as in practice but four items with title + short text + fake price/time.
4. Footer: two divs — left “Contact”, right “© 2026”.
5. Test resize behavior.

Starter snippets (copy-paste, then edit)

CSS snippet

```
.hero {
  display: flex;
  align-items: center;
  justify-content: center;
  min-height: 160px;
  background: #2c5282;
  color: #fff;
}
footer.site-footer {
  display: flex;
  justify-content: space-between;
  padding: 1rem 1.25rem;
  background: #edf2f7;
}
```

If it does not work

- Footer columns stack unexpectedly → check width of children; use `flex-wrap` if needed.

What to show the teacher (Done criteria)

Done when

- Four cards wrap cleanly
- Footer spaced left/right

PRACTICE 07 — CSS Grid + media queries

Timebox: ≈50–55 minutes · Folder: `CSE6624\_W07\_Practice\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Create a responsive card layout with CSS Grid and a media query that changes columns on narrow screens.

Deliverable checklist

- .cards uses display:grid with grid-template-columns
- gap set
- @media (max-width: 600px) reduces to one column
- viewport meta present

Click-by-click steps

1. HTML with three or more .card children inside .cards.
2. CSS: .cards { display: grid; grid-template-columns: repeat(3, 1fr); gap: 1rem; }.
3. Add media query for max-width 600px → grid-template-columns: 1fr;
4. DevTools device toolbar (Ctrl+Shift+M) → test 375px width.
5. Confirm mobile-first idea: narrow gets one column.

Starter snippets (copy-paste, then edit)

CSS snippet

```
.cards {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1rem;
  padding: 1rem;
}
.card {
  background: #fff;
  border: 1px solid #cbd5e0;
  padding: 1rem;
}
@media (max-width: 600px) {
  .cards {
    grid-template-columns: 1fr;
  }
}
```

If it does not work

- Media query “not working” → check viewport meta; check you resized below 600px; check braces.
- Grid ignored → typos in display:grid; ensure parent is .cards.

What to show the teacher (Done criteria)

Done when

- 3 columns on wide screen
- 1 column in device mode ~375px

LAB 07 — Landing page: grid layout + responsive polish

Timebox: ≈55–60 minutes · Folder: `CSE6624\_W07\_Lab\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Design a simple landing page: header, hero, grid features, footer. Include at least two breakpoints.

Deliverable checklist

- Grid areas OR multiple grid sections
- Breakpoints at ~600px and ~900px (or similar)
- Images (if any) max-width: 100%; height: auto
- Readable on phone width
- README lists your breakpoints

Click-by-click steps

1. Topic: “CSE6624 Student Landing” or your project idea preview.
2. Structure: header / hero / features grid (3) / footer.
3. Desktop: 3-column features; tablet: 2; mobile: 1.
4. Add `img { max-width: 100%; height: auto; }` if you include images.
5. Test 375 / 768 / 1200 widths in DevTools.
6. README: write “Breakpoints: ...”.

Starter snippets (copy-paste, then edit)

CSS snippet

```
.features {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 1rem;  
}  
@media (max-width: 900px) {  
  .features { grid-template-columns: repeat(2, 1fr); }  
}  
@media (max-width: 600px) {  
  .features { grid-template-columns: 1fr; }  
  .hero { padding: 1rem; }  
}
```

If it does not work

- Content overflow → check long words / fixed widths; prefer max-width and relative units.

What to show the teacher (Done criteria)

Done when

- Two breakpoints demonstrated
- Landing looks intentional (not random boxes only)

PRACTICE 08 — Project workshop checkpoint (milestone)

Timebox: ≈50–60 minutes · Folder: `CSE6624\_W08\_Practice\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Start (or continue) your Attestation 1 mini-site: choose topic, create 2 pages, shared nav, external CSS with Flex or Grid.

Deliverable checklist

- Topic written in README (one sentence)
- index.html + second page (e.g. tips.html or about.html)
- Shared nav both ways
- styles.css with Flex and/or Grid
- At least one image with alt OR one table/form started

Click-by-click steps

1. Read Attestation 1 brief (later chapter in this workbook) for 3 minutes.
2. Pick a campus topic you care about (club, dorm guide, course tips).
3. Scaffold folders/files: index.html, about.html (or tips.html), css/styles.css, images/, README.md.
4. Build semantic header/nav/main/footer on both pages.
5. Add layout with Flex or Grid from weeks 6–7.
6. Write real text (not lorem only) — at least 80 words total across pages.
7. Live Server check both pages.

Starter snippets (copy-paste, then edit)

TEXT snippet

```
README.md template
Name:
Group:
Topic:
Layout approach (Flex / Grid / both):
How to open: Live Server on index.html
AI used: yes/no – for what:
```

If it does not work

- No idea for topic → use “First-year Saturday survival guide for GUK714”.
- Pages look inconsistent → copy nav HTML carefully; share one CSS file.

What to show the teacher (Done criteria)

Done when

- Two linked pages open
- README topic sentence
- You can say what remains for Lab / attestation polish

LAB 08 — Milestone polish + peer checklist

Timebox: ≈55–60 minutes · Folder: `CSE6624\_W08\_Lab\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Push the mini-site toward Attestation 1 quality: media query, form or table, peer self-checklist filled.

Deliverable checklist

- Media query for narrow screens
- Form OR table included and working
- Images have alt (if present)
- Self-checklist file CHECKLIST.md completed with yes/no
- Zip-ready folder structure

Click-by-click steps

1. Continue from Practice files (copy into Lab folder or keep improving one project tree as instructed in class).
2. Add @media rules; test at 375px.
3. Add a contact form or schedule table (Week 04 skills).
4. Create CHECKLIST.md from the template below; mark each line yes/no honestly.
5. Fix any “no” you can in the remaining time.
6. Prepare questions for the teacher about attestation submission channel.

Starter snippets (copy-paste, then edit)

TEXT snippet

```
# Attestation 1 self-checklist
- [ ] 2–3 HTML pages with working nav
- [ ] Semantic landmarks
- [ ] External CSS
- [ ] Flex and/or Grid
- [ ] Media query
- [ ] Form or table
- [ ] Image alt texts
- [ ] README with name/group/how to open
- [ ] No React/Node
- [ ] I can explain my CSS layout choice
```

If it does not work

- Running out of time → prioritize nav + layout + README over decoration.

What to show the teacher (Done criteria)

Done when

- CHECKLIST.md with honest marks
- Mobile width demo
- Form or table visible

Attestation 1 — what to bring (after Week 08)

Official brief lives also in course-pack Attestation_1/ · Soft schedule follows university calendar

Product

A **responsive informational mini-site** (2–3 pages) about a campus topic of your choice (club, dorm guide, course tips, Saturday student map).

Must include

- Semantic HTML landmarks
- Internal navigation between pages
- Flexbox and/or Grid layout
- Media query for narrow screens
- Form or table (at least one)
- Images with alt text (if images used)
- External CSS file(s)

Must NOT include

- React, Node, npm apps
- Copied Bootstrap-only pages with no custom understanding (prefer your CSS for A1; Bootstrap arrives Week 09)

What to bring / submit

- Zip of your project folder
- README.md: name, group, how to open with Live Server
- Laptop ready to open the project in VS Code during check
- Ability to change one CSS property live when asked
- AI disclosure line if AI was used

Defence-style questions you should rehearse

- Where is your breakpoint?
- Why Flex vs Grid here?
- Show the form labels / table headers
- Open the site at phone width

PRACTICE 09 — Bootstrap 5 via CDN

Timebox: ≈50–55 minutes · Folder: `CSE6624\_W09\_Practice\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Build a page using Bootstrap 5 CDN: navbar + container + responsive card columns. Learn to read Bootstrap docs enough to copy a component safely.

Deliverable checklist

- Bootstrap 5.3 CSS CDN linked in <head>
- Navbar (can start from docs example, then customize text)
- container with a row of 3 cards (col-md-4)
- Page looks Bootstrap-styled without your own big CSS file (small custom CSS optional)

Click-by-click steps

1. Open Bootstrap docs: <https://getbootstrap.com/docs/5.3/getting-started/introduction/>.
2. Create index.html; paste CDN <link> for CSS from docs (jsDelivr).
3. Open Navbar examples: <https://getbootstrap.com/docs/5.3/components/navbar/> — copy a simple navbar; change brand to your name.
4. Add <div class="container py-4"> with <div class="row g-3"> and three <div class="col-md-4"> cards.
5. Cards: <https://getbootstrap.com/docs/5.3/components/card/>.
6. Live Server — resize to see columns stack.
7. Optional: add bootstrap.bundle.min.js before </body> if you use navbar toggler.

Starter snippets (copy-paste, then edit)

HTML snippet

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Week 09 Bootstrap Practice</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
        rel="stylesheet"
        integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh0JMhY6hW+ALEwIH"
        crossorigin="anonymous" />
</head>
<body class="bg-light">
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container">
      <a class="navbar-brand" href="#">Your Name</a>
    </div>
  </nav>
  <div class="container py-4">
    <h1 class="mb-4">Bootstrap practice</h1>
    <div class="row g-3">
      <div class="col-md-4"><div class="card h-100"><div class="card-body">Card A</div></div></div>
      <div class="col-md-4"><div class="card h-100"><div class="card-body">Card B</div></div></div>
      <div class="col-md-4"><div class="card h-100"><div class="card-body">Card C</div></div></div>
    </div>
  </div>
</body>
</html>
```

If it does not work

- Unstyled page → CDN blocked offline; need network OR download CSS locally.
- Integrity mismatch → copy the exact link tag from current Bootstrap docs.

- Columns not stacking → ensure col-md-4 and viewport meta exist.

What to show the teacher (Done criteria)

Done when

- Bootstrap look (not unstyled HTML)
- 3 cards in a row on desktop
- You know which classes came from Bootstrap

Tip — CDN not loading

If the classroom has no internet: download bootstrap.min.css to a USB stick beforehand and link locally href="css/bootstrap.min.css".

LAB 09 — Bootstrap landing: navbar + cards + form

Timebox: ≈55–60 minutes · Folder: `CSE6624\_W09\_Lab\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Compose a small landing page with navbar, hero (jumbotron-like), card row, and a Bootstrap-styled form.

Deliverable checklist

- Navbar with ≥2 links
- Hero section with btn btn-primary
- Card row
- Form using form-control and btn
- README

Click-by-click steps

1. Reuse CDN setup from Practice.
2. Add a hero: p-5 mb-4 bg-primary text-white rounded-3 with heading + button.
3. Add form with email + message using Bootstrap form classes.
4. Wire navbar anchors to #features and #contact sections.
5. Test responsive navbar (add toggler + bootstrap.bundle.min.js if using collapse).

Starter snippets (copy-paste, then edit)

HTML snippet

```
<section class="container my-4" id="contact">
  <h2>Contact</h2>
  <form class="row g-3" method="get" action="#">
    <div class="col-md-6">
      <label class="form-label" for="email">Email</label>
      <input class="form-control" id="email" type="email" required />
    </div>
    <div class="col-12">
      <label class="form-label" for="msg">Message</label>
      <textarea class="form-control" id="msg" rows="3"></textarea>
    </div>
    <div class="col-12">
      <button class="btn btn-primary" type="submit">Send</button>
    </div>
  </form>
</section>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
  integrity="sha384-YvpcrYf0tY3LHB60NNkmXc5s9fDVZLESaAA55NDz0xhy9GkcIdslK1eN7N6jIeHz"
  crossorigin="anonymous"></script>
```

If it does not work

- Toggler button does nothing → include bootstrap.bundle.min.js and matching data-bs attributes from docs.

What to show the teacher (Done criteria)

Done when

- Hero + cards + form on one page
- Form controls look Bootstrap-styled

PRACTICE 10 — JavaScript basics (console + tip calc snippet)

Timebox: ≈50–55 minutes · Folder: `CSE6624\_W10\_Practice\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Write your first intentional JS: variables, numbers, strings, console.log, and a tiny tip calculator on the page.

Deliverable checklist

- index.html + styles.css + main.js linked
- Console shows at least two console.log lines on load
- Inputs for bill and tip percent; button calculates
- Output area shows tip amount and total
- No unexpected red errors in Console

Click-by-click steps

1. Create files index.html, styles.css, main.js.
2. HTML: two number inputs, button, `<pre id="out">` or `<p id="out">`.
3. In main.js: log a hello string and typeof of a number.
4. Add click listener: read values with `Number(...)`, compute tip and total, write to out.
5. Open DevTools Console before clicking — watch logs.
6. Test bill=1000, tip=10 → tip 100, total 1100.

Starter snippets (copy-paste, then edit)

HTML snippet

```
<h1>Tip calculator</h1>
<label>Bill <input id="bill" type="number" min="0" step="0.01" value="1000" /></label>
<label>Tip % <input id="tip" type="number" min="0" value="10" /></label>
<button id="btn" type="button">Calculate</button>
<pre id="out">Output...</pre>
<script src="main.js"></script>
```

JAVASCRIPT snippet

```
console.log("W10 practice loaded");
console.log("typeof 42 =>", typeof 42);

const billEl = document.getElementById("bill");
const tipEl = document.getElementById("tip");
const out = document.getElementById("out");
const btn = document.getElementById("btn");

btn.addEventListener("click", function () {
  const bill = Number(billEl.value);
  const tipPct = Number(tipEl.value);
  if (Number.isNaN(bill) || Number.isNaN(tipPct)) {
    out.textContent = "Please enter valid numbers.";
    return;
  }
  const tipAmount = bill * (tipPct / 100);
  const total = bill + tipAmount;
  out.textContent = "Tip: " + tipAmount.toFixed(2) + "\nTotal: " + total.toFixed(2);
});
```

If it does not work

- NaN results → inputs empty or non-numeric; use `Number()` and validate.
- Nothing happens → check btn id; script path; look at Console errors.

- Using `prompt()` only → OK for experiment, but page UI is required for Done.

What to show the teacher (Done criteria)

Done when

- Calculate works for 1000 / 10%
- Console logs on load
- You explain what `Number()` does

LAB 10 — Tip calculator page (polished) + grade average tool

Timebox: ≈55–60 minutes · Folder: `CSE6624\_W10\_Lab\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Polish the tip calculator UI and add a second mini-tool: average of three grades entered in inputs.

Deliverable checklist

- Styled tip calculator (CSS)
- Second tool: 3 grade inputs → average button → output
- Guard against empty/invalid input with a user-visible message
- README explains both tools in two bullets

Click-by-click steps

1. Copy practice as starting point; improve layout with simple CSS.
2. Add section “Grade average” with three inputs g1 g2 g3.
3. Write function `average3(a,b,c)` in `main.js` and use it.
4. Show message if any value is missing.
5. Test several numbers; confirm Console clean.

Starter snippets (copy-paste, then edit)

JAVASCRIPT snippet

```
function average3(a, b, c) {  
  return (a + b + c) / 3;  
}  
// Example wiring:  
// const avg = average3(Number(g1.value), Number(g2.value), Number(g3.value));
```

If it does not work

- Integer division confusion → JS number division is float; use `toFixed(2)` for display.

What to show the teacher (Done criteria)

Done when

- Both tools work
- Invalid input shows a clear message

PRACTICE 11 — Functions, arrays, objects

Timebox: ≈50–55 minutes · Folder: `CSE6624\_W11\_Practice\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Store campus clubs as an array of objects and render their names into a list with a function.

Deliverable checklist

- `const clubs = [ { name, room }, ... ]` with ≥ 3 objects
- `function renderClubs(list) { ... }` builds `<li>` elements
- Button or on-load call renders into `<ul id="list">`
- `console.table(clubs)` used once

Click-by-click steps

1. Create `index.html` with `<ul id="list"></ul>` and optional filter input.
2. In `main.js` define `clubs` array of objects.
3. Write `renderClubs(list)` that clears list and appends `li` `textContent = club.name + " — " + club.room`.
4. Call `renderClubs(clubs)` on load.
5. In Console type `clubs` and press Enter — inspect the array.
6. Run `console.table(clubs)`.

Starter snippets (copy-paste, then edit)

JAVASCRIPT snippet

```
const clubs = [
  { name: "Robotics", room: "GUK201" },
  { name: "Debate", room: "GUK105" },
  { name: "Web Club", room: "GUK714" },
];

const list = document.getElementById("list");

function renderClubs(items) {
  list.innerHTML = "";
  for (const club of items) {
    const li = document.createElement("li");
    li.textContent = club.name + " — " + club.room;
    list.appendChild(li);
  }
}

renderClubs(clubs);
console.table(clubs);
```

If it does not work

- `undefined` — `undefined` → object property names must match what you read.
- `innerHTML` with untrusted input is risky later — for this week `textContent` on `li` is safer.

What to show the teacher (Done criteria)

Done when

- Three clubs visible
- `console.table` demonstrated
- You explain what an object property is

LAB 11 — Filterable club directory

Timebox: ≈55–60 minutes · Folder: `CSE6624\_W11\_Lab\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Add a text input that filters the clubs array by name (case-insensitive) and re-renders.

Deliverable checklist

- ≥5 club objects
- Filter input filters as you type (input event) OR on button
- Empty filter shows all
- README describes the data shape

Click-by-click steps

1. Extend Practice data to 5+ clubs.
2. Add `<input id="q" placeholder="Filter by name" />`.
3. Listen to input event; filter with `clubs.filter(c => c.name.toLowerCase().includes(q))`.
4. Pass filtered array to `renderClubs`.
5. Test “web”, “WEB”, empty string.

Starter snippets (copy-paste, then edit)

JAVASCRIPT snippet

```
const qEl = document.getElementById("q");
qEl.addEventListener("input", function () {
  const q = qEl.value.trim().toLowerCase();
  const filtered = clubs.filter(function (c) {
    return c.name.toLowerCase().includes(q);
  });
  renderClubs(filtered);
});
```

If it does not work

- Filter case-sensitive accidentally → always `toLowerCase` both sides.
- Nothing re-renders → confirm you call `renderClubs` after filter.

What to show the teacher (Done criteria)

Done when

- Typing filters the list live
- Five clubs in source data

PRACTICE 12 — DOM and events

Timebox: ≈50–55 minutes · Folder: `CSE6624\_W12\_Practice\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Practice `createElement`, `appendChild`, and click/submit events by building a tiny counter and a note adder.

Deliverable checklist

- Counter button increments a number on the page
- Form submit adds a note to a list without page reload (`preventDefault`)
- Delete button on each note OR a clear-all button

Click-by-click steps

1. HTML: `<p>Count: <span id="count">0</span></p><button id="inc">+1</button>`.
2. HTML: `form#form` with `input#item` and `submit`; `ul#list`.
3. JS: `inc click → count.textContent = Number(...) + 1`.
4. JS: `form submit → e.preventDefault(); create li; append; clear input`.
5. Add clear-all button that sets `list.innerHTML = ""`.

Starter snippets (copy-paste, then edit)

JAVASCRIPT snippet

```
const countEl = document.getElementById("count");
document.getElementById("inc").addEventListener("click", function () {
  countEl.textContent = String(Number(countEl.textContent) + 1);
});

const form = document.getElementById("form");
const item = document.getElementById("item");
const list = document.getElementById("list");

form.addEventListener("submit", function (e) {
  e.preventDefault();
  const text = item.value.trim();
  if (!text) return;
  const li = document.createElement("li");
  li.textContent = text;
  list.appendChild(li);
  item.value = "";
  item.focus();
});
```

If it does not work

- Page reloads on submit → missing `e.preventDefault()`.
- Duplicate ids → ids must be unique.

What to show the teacher (Done criteria)

Done when

- Counter works
- Add note without reload
- You point to `preventDefault` in code

LAB 12 — Todo-lite with DOM only

Timebox: ≈55–60 minutes · Folder: `CSE6624\_W12\_Lab\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Build a todo list: add, toggle done (click to strikethrough), delete item. No frameworks.

Deliverable checklist

- Add via form
- Click item toggles class done (CSS line-through)
- Delete button per item stops toggling (stopPropagation)
- Empty-state message when list is empty

Click-by-click steps

1. Design HTML shell with form + ul.
2. CSS: .done { text-decoration: line-through; opacity: 0.6; }.
3. On add: li contains span.text and button.del.
4. li click toggles .done on the li.
5. button.del click: e.stopPropagation(); li.remove(); update empty state.
6. Test carefully.

Starter snippets (copy-paste, then edit)

JAVASCRIPT snippet

```
li.addEventListener("click", function () {
  li.classList.toggle("done");
});
delBtn.addEventListener("click", function (e) {
  e.stopPropagation();
  li.remove();
  updateEmptyState();
});
```

If it does not work

- Delete also toggles done → need stopPropagation on delete click.
- Memory of items lost on refresh → OK this week; localStorage is Week 13.

What to show the teacher (Done criteria)

Done when

- Add / toggle / delete all work
- Empty state appears when last item removed

PRACTICE 13 — localStorage basics

Timebox: ≈50–55 minutes · Folder: `CSE6624\_W13\_Practice\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Persist a simple string or JSON array in localStorage so data survives refresh.

Deliverable checklist

- Save button writes to localStorage
- On load, page reads and displays saved value
- Clear button removes the key
- DevTools → Application → Local Storage shows your key

Click-by-click steps

1. HTML: input, Save, Clear, output.
2. Pick a key name like cse6624-w13-note.
3. Save: localStorage.setItem(key, input.value).
4. Load: const v = localStorage.getItem(key); show it.
5. Clear: localStorage.removeItem(key).
6. F12 → Application → Local Storage → your origin → verify.
7. Refresh page — value still there until Clear.

Starter snippets (copy-paste, then edit)

JAVASCRIPT snippet

```
const KEY = "cse6624-w13-note";
const input = document.getElementById("input");
const out = document.getElementById("out");

function show() {
  out.textContent = localStorage.getItem(KEY) || "(empty)";
}
show();

document.getElementById("save").addEventListener("click", function () {
  localStorage.setItem(KEY, input.value);
  show();
});
document.getElementById("clear").addEventListener("click", function () {
  localStorage.removeItem(KEY);
  show();
});
```

If it does not work

- Storage null → wrong key spelling; check Application panel.
- file:// quirks → use Live Server (http).

What to show the teacher (Done criteria)

Done when

- Save → refresh → still there
- Application panel shows key
- Clear works

LAB 13 — fetch local JSON + render cards (+ optional save)

Timebox: ≈55–60 minutes · Folder: `CSE6624\_W13\_Lab\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Create data.json, fetch it with fetch(), render cards. Optionally save favorite id to localStorage.

Deliverable checklist

- data.json with array of ≥3 objects
- fetch('./data.json') then response.json()
- Cards rendered into the DOM
- Error message if fetch fails
- README notes that Live Server is required for fetch

Click-by-click steps

1. Create data.json like [{"id":1,"title":"Library","blurb":"..."}, ...].
2. async function load() { try { const res = await fetch('./data.json'); ... } catch(e) {...} }.
3. Map items to card elements.
4. Optional: button “Favorite” stores id in localStorage.
5. Confirm Network tab shows data.json 200.

Starter snippets (copy-paste, then edit)

JSON snippet

```
[
  {
    "id": 1, "title": "Library", "blurb": "Quiet floors on exam week."
  },
  {
    "id": 2, "title": "Cafeteria", "blurb": "Try the tea after lab."
  },
  {
    "id": 3, "title": "GUK714", "blurb": "CSE6624 Saturday home."
  }
]
```

JAVASCRIPT snippet

```
async function load() {
  const root = document.getElementById("list");
  try {
    const res = await fetch("./data.json");
    if (!res.ok) throw new Error("HTTP " + res.status);
    const items = await res.json();
    root.innerHTML = "";
    for (const item of items) {
      const li = document.createElement("li");
      li.innerHTML = "<strong>" + item.title + "</strong>: " + item.blurb;
      root.appendChild(li);
    }
  } catch (err) {
    root.textContent = "Could not load data.json – use Live Server. " + err.message;
    console.error(err);
  }
}
load();
```

If it does not work

- fetch fails on file:// → Live Server mandatory.
- JSON parse error → validate commas/quotes in data.json.

What to show the teacher (Done criteria)

Done when

- Cards from JSON
- Network 200 for data.json
- Failure message demo if teacher renames file

PRACTICE 14 — Accessibility quick wins

Timebox: ≈50–55 minutes · Folder: `CSE6624\_W14\_Practice\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Audit a page for alt text, labels, heading order, contrast, and focus visibility. Fix the easy issues.

Deliverable checklist

- All images have meaningful alt (or alt="" if decorative — justify in README)
- Every input has a label
- One h1 only; heading levels do not skip wildly
- :focus-visible outline styles for links/buttons
- A11Y_NOTES.md lists 5 checks you performed

Click-by-click steps

1. Read MDN a11y intro briefly: <https://developer.mozilla.org/en-US/docs/Learn/Accessibility>.
2. Take any previous project page (or starter) and copy into this week's folder.
3. Fix missing labels/alts.
4. Add CSS: a:focus-visible, button:focus-visible { outline: 3px solid #dd6b20; outline-offset: 2px; }.
5. Check color contrast roughly (dark text on light bg).
6. Keyboard-only: Tab through interactive elements — can you see focus?
7. Write A11Y_NOTES.md with 5 bullets of what you fixed or verified.

Starter snippets (copy-paste, then edit)

CSS snippet

```
a:focus-visible,  
button:focus-visible,  
input:focus-visible,  
select:focus-visible,  
textarea:focus-visible {  
  outline: 3px solid #dd6b20;  
  outline-offset: 2px;  
}
```

If it does not work

- Focus outline removed globally (outline: none) without replacement → restore focus-visible styles.
- alt="image" is not meaningful — describe content.

What to show the teacher (Done criteria)

Done when

- Tab focus visible
- A11Y_NOTES.md present
- Labeled form fields

LAB 14 — Polish pass for your project

Timebox: ≈55–60 minutes · Folder: `CSE6624\_W14\_Lab\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Apply accessibility + performance + visual polish to your evolving course project (Attestation 2 / exam candidate).

Deliverable checklist

- Compressed / reasonably sized images
- Consistent spacing and typography
- No Console errors
- README updated with feature list
- Screenshots folder with 1 desktop + 1 mobile capture

Click-by-click steps

1. Copy your best multi-page project into the lab folder (or work in place if teacher says so).
2. Fix Console errors first.
3. Resize large images; ensure `img { max-width:100%; }`.
4. Unify font sizes and colors (limit palette to ~3 colors + neutrals).
5. Run through Week 08 checklist + Week 14 a11y notes.
6. Capture screenshots; put in screenshots/.
7. Update README feature list for defence.

Starter snippets (copy-paste, then edit)

TEXT snippet

```
## Features
- ...
## How to run
Live Server on index.html
## Accessibility notes
- ...
## AI disclosure
- ...
```

If it does not work

- Too many unfinished experiments → delete dead files before defence.

What to show the teacher (Done criteria)

Done when

- Screenshots present
- Console clean
- README feature list

PRACTICE 15 — Final project defence prep (dry run)

Timebox: ≈50–55 minutes · Folder: `CSE6624\_W15\_Practice\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Rehearse a 3–4 minute demo script and prepare answers to common defence questions.

Deliverable checklist

- DEMO_SCRIPT.md (bullet timing: intro, pages, JS feature, a11y, closing)
- Answers draft for 5 defence questions (see end of workbook)
- Project opens in < 30 seconds on your laptop
- You can change one color and one string live

Click-by-click steps

1. Open your exam project candidate.
2. Write DEMO_SCRIPT.md with timed bullets (total ≤4 minutes).
3. Practice speaking once with a timer on your phone.
4. Write ANSWERS.md with short answers to questions 1–5 from the defence bank.
5. Practice: teacher points at a function — you explain without scrolling randomly.
6. Fix anything that broke during rehearsal.

Starter snippets (copy-paste, then edit)

TEXT snippet

```
# DEMO_SCRIPT.md
0:00 Intro – name, topic, folder structure
0:40 Page walkthrough (Home → ...)
1:40 Layout (Flex/Grid) + responsive proof
2:20 JavaScript feature demo
3:00 localStorage or fetch
3:20 Accessibility highlight
3:40 Closing – what I would improve
```

If it does not work

- Demo too long → cut decorative slides; show working software, not PowerPoint-only.

What to show the teacher (Done criteria)

Done when

- DEMO_SCRIPT.md
- Live color change
- Timed rehearsal done once

LAB 15 — Final packaging: zip, README, live link, defence notes

Timebox: ≈55–60 minutes · Folder: `CSE6624\_W15\_Lab\_Surname\_Name` · Soft grading · AI OK if you can explain & change

Goal

Package the final submission: clean zip, complete README, optional GitHub Pages link, defence notes printed or PDF.

Deliverable checklist

- Zip named CSE6624_Exam_Surname_Name.zip (or as announced)
- README complete (name, group, setup, features, AI disclosure)
- No node_modules or huge binaries
- DEFENCE_NOTES.md with 3 bullets “what JS does”
- Optional: GitHub repo link / Pages URL

Click-by-click steps

1. Delete unused scratch files; keep source only.
2. Confirm relative paths work on a clean Live Server open.
3. Optional GitHub upload: create repo on <https://github.com/new>, push via Git as taught, enable Pages (<https://docs.github.com/en/pages>) if time allows.
4. Write DEFENCE_NOTES.md — 3 bullets on JS behaviour.
5. Zip the folder; upload per instructor announcement.
6. Sleep / rest — do not rewrite architecture the night before.

Starter snippets (copy-paste, then edit)

TEXT snippet

```
# README (exam)
Name:
Group:
Contact email:
How to open: install nothing beyond browser + Live Server; open index.html
Pages:
Features:
JS interactions:
Storage / fetch:
AI disclosure:
Known limitations:
```

If it does not work

- Zip contains wrong parent folder → zip the project root that contains index.html.
- Pages 404 → check branch/folder settings in GitHub Pages docs.

What to show the teacher (Done criteria)

Done when

- Zip ready
- README complete
- DEFENCE_NOTES.md with 3 JS bullets

Attestation 2 — note (~Week 15)

Enhance your project (or a dedicated mini-app page) with **JavaScript behaviour**:

- DOM updates from events
- At least one array/object used as data
- localStorage OR fetch (local JSON OK)
- Basic a11y pass (labels, alt, focus visible)

Submit updated zip + README + 3 bullets “what JS does”. Be ready to modify a function live.

End-of-term / Exam project defence checklist

Product (from exam brief)

A **multi-page responsive static website** with light JavaScript interactivity. Suggested themes: campus guide, personal portfolio, club fair, study planner (front-end only).

Required features

- ≥ 3 HTML pages sharing navigation
- Semantic layout + consistent CSS (Flex/Grid)
- Responsive design (narrow screens usable)
- One form (validation attributes) and/or data table
- JavaScript: ≥ 2 interactions (menu toggle, filter, theme toggle, tip calc, todo-lite, ...)
- localStorage OR fetch of local JSON for one feature
- README: name, setup, feature list, AI disclosure if used
- Basic a11y: alt, labels, contrast effort

Out of scope

- React
- Node / Express
- Databases
- PHP

Submission

- Zip + optional GitHub Pages link
- Defence per exam schedule

Defence question bank (rehearse aloud)

1. Walk me through your folder structure.
2. Where is your responsive breakpoint defined?
3. Why did you choose Flexbox vs Grid here?
4. What happens when this button is clicked? (point to code)
5. Change this color/spacing live.
6. Add a console.log when the form submits.
7. What would you improve with one more day?
8. Did you use AI? Show one part you rewrote.
9. How do you know your image paths work on another computer?
10. Which accessibility fix are you proud of?

Day-of-defence kit

- Laptop charged + charger
- VS Code + Live Server working offline for your files
- Zip uploaded / USB backup
- DEMO_SCRIPT.md open
- Water, calm voice, honest AI disclosure

Appendix — Quick URL list

- VS Code: <https://code.visualstudio.com/>
- Live Server extension: <https://marketplace.visualstudio.com/items?itemName=ritwickdey.LiveServer>
- Prettier: <https://marketplace.visualstudio.com/items?itemName=esbenp.prettier-vscode>
- ESLint: <https://marketplace.visualstudio.com/items?itemName=dbaumer.vscode-eslint>
- Git: <https://git-scm.com/>
- GitHub signup: <https://github.com/signup>
- Chrome: <https://www.google.com/chrome/>
- Satbayev University: <https://satbayev.university>
- MDN Learn HTML: https://developer.mozilla.org/en-US/docs/Learn/HTML/Introduction_to_HTML
- MDN Accessibility: <https://developer.mozilla.org/en-US/docs/Learn/Accessibility>
- Bootstrap 5.3 docs: <https://getbootstrap.com/docs/5.3/getting-started/introduction/>
- GitHub Pages docs: <https://docs.github.com/en/pages>
- Picsum placeholder images: <https://picsum.photos/>

End of CSE6624 Practice & Lab Workbook · Autumn 2026–2027 · Temirlan Shoiynbek